



Operational Research

Linear Programming solver

By :

Amjaad Hawsawi
Norah Alkabkabi
Weaam Rashed

For this project, we built a lightweight Linear Programming solver in Python that reads your objective (maximize or minimize), the number of decision variables, and a matching set of equations. Under the hood, Gaussian elimination with partial pivoting is used to find the exact solution. Then, the best value of the objective function and each decision variable is printed out.

```
import numpy as np

def gaussian_elimination_with_pivoting(A, b):
    n = len(b)
    augmented = np.hstack([A, b.reshape(-1, 1)]).astype(float)

    for i in range(n):
        max_row = i + np.argmax(abs(augmented[i:, i]))
        if augmented[max_row, i] == 0:
            raise ValueError("Matrix is singular or system is inconsistent")
        augmented[[i, max_row]] = augmented[[max_row, i]]

        for j in range(i + 1, n):
            factor = augmented[j, i] / augmented[i, i]
            augmented[j, i:] -= factor * augmented[i, i:]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        if abs(augmented[i, i]) < 1e-12:
            raise ValueError("Zero diagonal element encountered during back substitution.")
        x[i] = (augmented[i, -1] - np.dot(augmented[i, i+1:n], x[i+1:n])) / augmented[i, i]

    return x

def solve_lp():
    print("\n--- Linear Programming Solver ---")

    obj_type = input("Enter objective type (max/min): ").lower()
    if obj_type not in ['max', 'min']:
        raise ValueError("Objective must be 'max' or 'min'")

    n_vars = int(input("Enter number of decision variables: "))
    if n_vars <= 0:
        raise ValueError("Number of variables must be positive")

    print("Enter objective function coefficients (e.g., for Z = 2x1 + 3x2 + x3, enter: 2 3 1):")
    obj_coeffs = np.array([float(x) for x in input().split()])
    if len(obj_coeffs) != n_vars:
        raise ValueError("Number of coefficients must match number of variables")

    n_constraints = int(input("Enter number of constraint equations: "))
    if n_constraints != n_vars:
        raise ValueError(
            f"For a unique solution, number of constraints ({n_constraints}) "
            f"must equal number of variables ({n_vars})."
        )

    print("Enter constraint coefficients row by row (space-separated, including RHS):")
    A = np.zeros((n_constraints, n_vars))
    b = np.zeros(n_constraints)
    for i in range(n_constraints):
        row = [float(x) for x in input(f"Row {i+1}: ").split()]
        if len(row) != n_vars + 1:
            raise ValueError("Each constraint must have coefficients + RHS")
        A[i, :] = row[:-1]
        b[i] = row[-1]

    try:
        solution = gaussian_elimination_with_pivoting(A, b)
    except ValueError as e:
        print(f"Error: {e}")
        return

    obj_value = np.dot(obj_coeffs, solution)

    print("\nResults:")
    print("Optimum solution:")
    print(f"Z = {obj_value:4f} ({'maximized' if obj_type == 'max' else 'minimized'})")
    print("Decision variable values:")
    for i, val in enumerate(solution, 1):
        print(f"x{i} = {val:4f}")

def main():
    while True:
        solve_lp()
        cont = input("\nWould you like to solve another LP problem? (yes/no): ").strip().lower()
        if cont != 'yes':
            print("Exiting LP Solver. Goodbye!")
            break

if __name__ == "__main__":
    main()
```

```

***
--- Linear Programming Solver ---
Enter objective type (max/min): max
Enter number of decision variables: 3
Enter objective function coefficients (e.g., for  $Z = 2x_1 + 3x_2 + x_3$ , enter: 2 3 1):
2 3 1
Enter number of constraint equations: 3
Enter constraint coefficients row by row (space-separated, including RHS):
Row 1: 25 5 1 106.8
Row 2: 64 8 1 177.2
Row 3: 144 12 1 279.2

Results:
Optimum solution:
 $Z = 60.7381$  (maximized)
Decision variable values:
 $x_1 = 0.2905$ 
 $x_2 = 19.6905$ 
 $x_3 = 1.0857$ 

Would you like to solve another LP problem? (yes/no): 

```

```

***
Would you like to solve another LP problem? (yes/no): yes

--- Linear Programming Solver ---
Enter objective type (max/min): max
Enter number of decision variables: 2
Enter objective function coefficients (e.g., for  $Z = 2x_1 + 3x_2 + x_3$ , enter: 2 3 1):
1 1
Enter number of constraint equations: 2
Enter constraint coefficients row by row (space-separated, including RHS):
Row 1: 1 1 3
Row 2: 3 -2 4

Results:
Optimum solution:
 $Z = 3.0000$  (maximized)
Decision variable values:
 $x_1 = 2.0000$ 
 $x_2 = 1.0000$ 

Would you like to solve another LP problem? (yes/no): 

```

```

Would you like to solve another LP problem? (yes/no): yes
⌂
--- Linear Programming Solver ---
Enter objective type (max/min): max
Enter number of decision variables: 3
Enter objective function coefficients (e.g., for  $Z = 2x_1 + 3x_2 + x_3$ , enter: 2 3 1):
1 1 1
Enter number of constraint equations: 3
Enter constraint coefficients row by row (space-separated, including RHS):
Row 1: 5 2 0 2
Row 2: 2 1 -1 0
Row 3: 2 3 -1 3

Results:
Optimum solution:
 $Z = 2.4000$  (maximized)
Decision variable values:
 $x_1 = -0.2000$ 
 $x_2 = 1.5000$ 
 $x_3 = 1.1000$ 

Would you like to solve another LP problem? (yes/no): no
Exiting LP Solver. Goodbye!

```